



Bitácora de Cambios — Atlas Gym Frontend

Este documento registra cada modificación realizada en el sistema, el motivo de negocio/técnico detrás de cada decisión y los archivos exactos afectados. Servirá de referencia tanto para el equipo de desarrollo como para reportar avances al cliente.



Sprint 1: Panel de Alumnos (Complejidad Crítica)

1. Edición de Alumnos (Botón y Modal)

- Orden y Requerimiento del Proyecto (Lucas / Cuaderno de Requerimientos):

“A todo lo que hagamos tenemos que armar un documento con los cambios, el por qué y el dónde se hicieron.”

*“Requerimiento base relevado del cliente: En el cuaderno manuscrito de especificaciones se pedía explícitamente en el Panel de Alumnos la acción de **"Editar"**, la cual faltaba en la plantilla inicial.”*

- ¿Qué se hizo?
- Se agregó un botón con ícono de lápiz (✎) en cada fila de la tabla de alumnos.
- Se construyó el componente `ModalEditarAlumno`, que se abre al hacer clic en dicho botón y viene pre-cargado con los datos actuales del alumno (Nombre, Plan, Email, Celular y Estado Activo/Inactivo).
- Se conectó la acción "Guardar cambios" directamente al store centralizado mediante la función `actualizarAlumno()`.
- ¿Por qué se hizo?
- **Requerimiento del cliente:** El sistema solo permitía agregar alumnos nuevos o eliminarlos; no había forma de corregir un número de teléfono, cambiar un plan o actualizar datos sin borrar y volver a crear al alumno desde cero.
- ¿Dónde se hizo?

- `app/alumnos/page.tsx` :
- Se importó el ícono `Pencil` de `lucide-react` .
- Se extrajo `actualizarAlumno` desde `useAppData()` .
- Se agregó el estado local `alumnoAEditar` .
- Se añadió el botón en la columna de acciones de la tabla.
- Se implementó el componente `ModalEditarAlumno` .

2. Detección e Inactividad Automática (+60 días)

- **Orden y Planteo del Problema (Lucas / Equipo Atlas):**

“Consulta y consenso operativo: ¿Cómo detecta el gimnasio si un alumno abandonó sin tener molinetes caros ni obligar al profesor a tildar a mano uno por uno?”

“Decisión adoptada: Automatizar el estado en base a la falta de pago (+60 días sin cuotas abonadas).”

- **¿Qué se hizo?**
- Se agregó el estado `"INACTIVO"` al tipo `EstadoCuenta` .
- Se programó una función lógica en `lib/types.ts` (`estadoCuentaDeAlumno`) que calcula automáticamente la inactividad: si un alumno lleva más de 60 días sin registrar ningún pago de cuota (o más de 60 días desde su fecha de alta sin pagos), el sistema lo clasifica como `Inactivo (+60d)` .
- Si el alumno abona una cuota nueva en Finanzas, el sistema lo reactiva automáticamente a `AL día` o `Pendiente` sin intervención manual.
- **¿Por qué se hizo?**
- **Problema operativo real:** El gimnasio no cuenta con lectores de huella o molinetes automáticos para registrar entradas. El cliente no tiene tiempo de cambiar manualmente el estado de cada persona uno por uno.
- **Solución de negocio:** La fuente de verdad más confiable que el dueño sí carga con regularidad son los pagos en Finanzas. Si alguien deja de pagar por 2 meses consecutivos, el sistema asume inteligentemente el abandono, ahorrándole al dueño horas de gestión manual.
- **¿Dónde se hizo?**
- `lib/types.ts` :
- Ampliación de `EstadoCuenta` : `"AL_DIA" | "PENDIENTE" | "MOROSO" | "INACTIVO"` .
- Actualización de la función `estadoCuentaDeAlumno(alumnoId, pagos, fechaAlta)` .
- Definición de etiquetas y estilos para el badge de inactivo (`bg-slate-100 text-slate-500 border-slate-200`).
- `lib/store.tsx` :

- Se actualizó `getEstadoCuenta` para pasar la fecha de alta del alumno como parámetro de cálculo.

3. Nuevas Columnas en la Tabla: "Rutina" y "Último ingreso"

- **Requerimiento del Cliente (Relevado del Cuaderno):**

"Listado: Rutina sí/no — último ingreso — cuota — estado — Editar"

- **¿Qué se hizo?**
- Se sumó la columna **"Rutina"**: Muestra una pastilla distintiva con ícono de mancuerna (`Asignada` en azul suave si tiene rutina asignada, o `Sin rutina` en gris neutro).
- Se sumó la columna **"Último ingreso"**: Muestra el tiempo transcurrido desde la última asistencia con formato amigable (`Hoy`, `Ayer`, `Hace X días` o `Sin registros` en cursiva).
- **¿Por qué se hizo?**
- Le da al dueño un paneo visual inmediato de a qué alumnos les falta armarles la rutina y quiénes asisten con regularidad.
- **¿Dónde se hizo?**
- `lib/types.ts`: Se añadió el campo opcional `tieneRutina?: boolean` en la interfaz `Alumno`.
- `lib/mock-data.ts`: Se completaron los datos mockeados con `tieneRutina` y se sumó un alumno inactivo de prueba (`Esteban Morales`, último pago en mayo de 2026).
- `app/alumnos/page.tsx`:
- Creación del helper `formatDiasIngreso(fecha: string)`.
- Nuevas cabeceras en la tabla y nuevas celdas con sus respectivos estilos y badges.
- Actualización del `colSpan={6}` para el estado vacío.

4. Cuarta Tarjeta de Métricas y Filtro Rápido

- **Objetivo de Gestión Comercial:**

"Definición de producto: Permitir al gimnasio segmentar a los alumnos que dejaron de asistir para poder enviarles un WhatsApp de reinscripción o promoción."

- **¿Qué se hizo?**
- Se rediseñó la cabecera de métricas para pasar de 3 a 4 tarjetas responsivas:
 1. **Total de alumnos** (Azul)
 2. **Al día** (Verde)
 3. **Morosos** (Rojo)

4. **Inactivos** (Gris / Pizarra) con ícono de usuario tachado (`UserX`).

- Se incorporó el botón `Inactivos (+60d)` en la barra de filtros rápidos sobre la tabla.
- **¿Por qué se hizo?**
- Permite al dueño o recepcionista filtrar con un solo clic a todos los clientes inactivos para lanzar campañas de reactivación/fidelización.
- **¿Dónde se hizo?**
- `app/alumnos/page.tsx` :
- Actualización de la métrica `inactivos` en el hook `useMemo` .
- Nueva tarjeta `MetricCard` con icono `UserX` .
- Nuevo botón en el array `FILTROS` .

5. Pantalla de Inicio de Sesión (`/login`)

- **Orden y Consulta del Usuario (Lucas):**

"El login estaría bueno que hagamos primero, no? pasa que no sé nada de apis yo como para buscar una que tenga todos los ejercicios"

- **¿Qué se hizo?**
 - Se construyó una pantalla de Login completa en la ruta `/login` .
 - Diseño 100% alineado con la marca Atlas: fondo oscuro con resplandor radial azul, tarjeta con efecto cristal (`backdrop-blur`), isotipo de Atlas y tipografía estilizada.
 - Selector de rol mediante pestañas interactivas: **"Dueño / Admin"** vs **"Alumno"** .
 - Formulario con validación, iconos de `Mail` y `Lock` , toggle para ver/ocultar contraseña (ojo `👁`) y checkbox de "Recordar sesión".
 - Caja de **Acceso Rápido Demo** con 2 botones de un solo clic (`👑 Admin` y `👤 Alumno`) para agilizar pruebas de navegación sin tener que tipear contraseñas cada vez.
 - Se actualizó `components/app-shell.tsx` para detectar la ruta `/login` y ocultar automáticamente la barra lateral (Sidebar).
 - **¿Por qué se hizo?**
 - **Requerimiento de negocio y seguridad:** El sistema contempla perfiles con accesos diferentes (los dueños administran todo el gimnasio, mientras que los alumnos consultan sus rutinas y pagos). Se necesita un punto de entrada visual antes de conectar la autenticación del backend.
 - **¿Dónde se hizo?**
 - `app/login/page.tsx` : Creación de la página y lógica de acceso rápido.
 - `components/app-shell.tsx` : Ocultamiento de la barra lateral en la ruta `/login` .
-

6. Vista Personalizada de Cuotas para el Alumno

- Orden y Observación del Usuario (Lucas):

““Quedó linda la prueba, después lo mejoramos. Pero noté que si entramos como alumno en finanzas es el mismo dashboard, para mí solo debería decir cuotas si entras como alumno y próximas cuotas””

- ¿Qué se hizo?

- Se integró la gestión de sesión activa (`usuarioActual` , `iniciarSesion` , `cerrarSesion`) en el store central (`lib/store.tsx`).
- En `components/sidebar.tsx` , el menú se adapta dinámicamente según el rol:
- Si ingresa un **Alumno**, se ocultan los módulos administrativos ("Inicio", "Alumnos", "Catálogo") y solo se le muestra **"Mis Cuotas"** y **"Mi Rutina"**, además de su propio nombre y avatar en el perfil inferior.
- En `app/finanzas/page.tsx` , se añadió la vista dedicada `Alumno`:
- Oculto totalmente la contabilidad general, ingresos del gimnasio y pagos de otros alumnos.
- Muestra tarjetas personales: Estado de su cuota (`Al día` / `Pendiente`), su Plan actual y fecha de próximo vencimiento (10 de cada mes).
- Sección de pago por transferencia con Alias copiable en 1 clic (`ATLAS.GYM.MP`) y botón directo para enviar el comprobante por WhatsApp.
- Tabla privada con únicamente el historial de pagos del alumno logueado.

- ¿Por qué se hizo?

- Privacidad y lógica de negocio:** Un alumno jamás debe tener visibilidad sobre la facturación global, ingresos ni los pagos del resto de los clientes del gimnasio. Su experiencia debe estar enfocada exclusivamente en saber cuánto debe, cuándo vence y cómo pagar.

- ¿Dónde se hizo?

- `lib/types.ts` : Definición de la interfaz `UsuarioSesion` .
- `lib/store.tsx` : Manejo de `usuarioActual` , `iniciarSesion` y persistencia de sesión.
- `components/sidebar.tsx` : Menú dinámico y perfil adaptado a Alumno vs Admin.
- `app/finanzas/page.tsx` : Componente `Alumno` y render condicional por rol.
- `app/login/page.tsx` : Conexión de login con `iniciarSesion()` .

7. Home Personalizado del Alumno y Videoteca en Menú

- Orden y Observación del Usuario (Lucas):

““Sigue diciendo panel para profesor arriba, nada que ver. Igual me gustó. Habría que idear la lógica de un home para alumnos y la videoteca debe poder verla””

"Acordate de anotar más que nada la lógica en ese documento además de lo que te pedí. Me gustó la lógica"

- **¿Qué se hizo?**

- Se corrigió el título global de la pestaña del navegador en `app/layout.tsx` de "ATLAS – Panel del Profesor" a "ATLAS GYM".
- Se implementó el componente `Header` en la raíz (`/`), que se activa automáticamente cuando el usuario logueado tiene rol `ALUMNO`:
- Saludo personalizado y motivacional (*¡Hola, {nombre}! Lista para romperla en tu entrenamiento de hoy!*).
- **Tarjeta "Mi Rutina de hoy"**: Muestra la rutina asignada ("Hipertrofia Nivel 2", grupo muscular del día, ejercicios) con botón directo para ver los ejercicios y series.
- **Tarjeta "Estado de mi cuota"**: Indicador rápido (*Al día / Pendiente*) con acceso a sus cuotas.
- **Tarjeta "Mi Constancia"**: Registro de asistencias y racha de entrenamientos del mes.
- **Sección "Videoteca Destacada"**: Miniaturas y accesos rápidos a videos de técnica de ejercicios (Press banca, Sentadillas, Remo con barra).
- **Sección "Horarios y Avisos"**: Información del gimnasio y recordatorios comunitarios.
- En `components/sidebar.tsx`:
- Se agregó el botón "Inicio" para los alumnos para que puedan volver a su Home cuando quieran.
- Se agregó la sección "Videoteca" (`/videoteca`) tanto para el Administrador como para el Alumno con icono de video.

- **¿Por qué se hizo? (Lógica de negocio y UX)**

- **Experiencia de usuario diferenciada**: El alumno que abre la app en el gimnasio desde su celular necesita ver inmediatamente qué le toca entrenar hoy, si su cuota está al día y cómo hacer bien los ejercicios para no lesionarse. No tiene sentido abrumarlo con números de facturación ni listas de otros alumnos.
- **Acceso universal a la Videoteca**: La videoteca es una herramienta de valor para el cliente del gimnasio (para consultar cómo se hace un ejercicio) y para el profesor/dueño (para cargar nuevos videos demostrativos).

- **¿Dónde se hizo?**

- `app/layout.tsx`: Corrección del `title` en los metadatos globales.
 - `app/page.tsx`: Componente `Header`, condicional de render por rol y saludo dinámico para administradores.
 - `components/sidebar.tsx`: Pestaña "Videoteca" e "Inicio" en los ítems de navegación de ambos roles.
-

8. Módulo Completo de Rutinas: Plantillas Genéricas, Días de Entrenamiento y Asignación a Alumnos

- **Orden y Consulta del Usuario (Lucas):**

""Bueno acá tengo yo un par de ideas pero mañana cuando hable con mis compañeros puede cambiar: El nombre de la rutina lo debe poner el dueño O debe ser "Día 1", "Día 2", etc... acordate que, los dueños pueden asignarle a cada alumno una rutina o armar rutinas genéricas con el nombre que ellos quieran, esas rutinas genéricas debense poder asignar a el alumno que ellos quieran también, ¿se entiende?""

""Vamos con rutinas. En el documento poné también la orden que te di yo o la consulta para llegar a la conclusión que se llegó para que ellos entiendan también""

- **Conclusión y Lógica de Negocio Adoptada:**

- Para satisfacer ambos requerimientos (libertad total para el dueño y orden claro para el alumno), se diseñó una **arquitectura jerárquica de 3 niveles**:

1. **Nivel 1 — Nombre General de la Rutina:** Lo define libremente el dueño/profesor según el objetivo (ej: "Hipertrofia Nivel 2", "Fuerza Básica 3 Días", "Adaptación Inicial", "Pérdida de Grasa").

2. **Nivel 2 — Días o Bloques:** Cada rutina se desglosa internamente en días específicos titulados de forma clara (ej: "Día 1 — Pecho y Tríceps", "Día 2 — Espalda y Bíceps", "Día 3 — Piernas y Hombros").

3. **Nivel 3 — Ejercicios del Día:** Cada día contiene sus ejercicios con: *Nombre, Series, Repeticiones, Descanso* (ej: 90s) y *Notas de técnica*.

- **Plantillas Genéricas vs Asignación:**

- Las rutinas se guardan en el sistema como plantillas maestras disponibles en el catálogo.
- Cualquier plantilla genérica se puede asignar a uno o varios alumnos desde un selector modal con un solo clic.
- A su vez, en la ficha individual de cada alumno (`/alumnos/[id]`) se visualiza su rutina activa y se le puede reasignar otra cuando progrese.
- Los alumnos, al entrar con su usuario o consultar `/rutinas` , ven exclusivamente la rutina que les fue asignada ("**Mi Rutina**") con sus días y ejercicios.

- **¿Qué se hizo a nivel de código?**

- **Tipos y Modelo de Datos (`lib/types.ts`):** Se crearon las interfaces `Ejercicio` , `DiaRutina` , `Rutina` y se enlazó el campo `rutinaId` opcional en la interfaz `Alumno` .
- **Mock Data & Store (`lib/mock-data.ts` y `lib/store.tsx`):**
 - Se cargaron rutinas completas de ejemplo con ejercicios reales y descansos.
 - Se crearon las funciones de store: `agregarRutina()` , `asignarRutinaAAlumno()` , `eliminarRutina()` y `getRutinaDeAlumno()` .

- **Panel de Rutinas** (`app/rutinas/page.tsx`):
 - **Para Administradores:** Listado de todas las rutinas con filtros por objetivo (*Todos, Hipertrofia, Fuerza, Definición*), vista desplegable de días y ejercicios, botón `+ Nueva Rutina` (con modal para ingresar nombre, objetivo y días) y botón `Asignar a Alumno` (con modal selector de alumnos).
 - **Para Alumnos** (): Vista limpia y enfocada donde el alumno ve solo su plan actual, los días de entrenamiento, series, repeticiones y consejos de su profesor.
 - **Ficha del Alumno** (`app/alumnos/[id]/page.tsx`):
 - Se reemplazó el cartel provisional de "Próximamente" por el visor interactivo de la rutina real asignada a ese alumno, mostrando cada día con sus ejercicios, series y pausas.
 - **¿Dónde se hizo?**
 - `lib/types.ts` : Modelos `Ejercicio` , `DiaRutina` , `Rutina` .
 - `lib/mock-data.ts` : Datos iniciales de rutinas y ejercicios.
 - `lib/store.tsx` : Estado global de rutinas y acciones de asignación.
 - `app/rutinas/page.tsx` : Pantalla completa de gestión de rutinas y vista del alumno.
 - `app/alumnos/[id]/page.tsx` : Visualizador de rutina activa en la ficha del alumno.

9. Módulo Completo de Videoteca: Biomecánica, Tutoriales y Filtros Musculares

- **Orden y Consulta del Usuario (Lucas):**

""habría que idear la lógica de un home para alumnos y la videoteca debe poder verla""

""Con qué crees que sería apropiado seguir? después vamos puliendo detalles""

- **Lógica de Negocio y Experiencia de Usuario (UX):**
 - En un gimnasio de pesas o entrenamiento funcional, muchos alumnos (sobre todo principiantes o intermedios) sienten vergüenza o inseguridad al no recordar exactamente cómo se ejecuta un ejercicio indicado en su rutina.
 - Para solucionar esto sin depender de APIs externas pagas o complejas, se construyó una **Videoteca interna integrada en ATLAS**:
- **Para el Alumno:**
 - Puede consultar en cualquier momento la técnica de los ejercicios divididos por grupo muscular (*Pecho, Espalda, Piernas, Hombros, Brazos, Core*).
 - Puede buscar por nombre de ejercicio o concepto clave (ej: `*"remo"`, `*"sentadilla"`, `*"escapular"`).

- Al tocar una tarjeta de video, se abre un **reproductor modal interactivo** con el video embebido, descripción anatómica y una lista de **tips clave de ejecución** (ej: alineación, respiración, posición de espalda) para entrenar con técnica estricta y sin riesgo de lesión.
- **Para el Administrador / Entrenador:**
- Cuenta con el botón `+ Nuevo Video` para incorporar tutoriales propios o de YouTube/Vimeo indicando el ejercicio, grupo muscular, nivel, duración estimada y puntos clave.
- Puede eliminar videos obsoletos con confirmación de seguridad.
- **¿Qué se hizo a nivel de código?**
- **Tipos y Modelo (`lib/types.ts`):** Creación de la interfaz `VideoTecnica` con atributos de título, grupo muscular, nivel, duración, URL y consejos biomecánicos.
- **Datos Iniciales y Store (`lib/mock-data.ts` y `lib/store.tsx`):**
- Lista `VIDEOS_TECNICA MOCK` con 6 ejercicios fundamentales (Press banca, Sentadillas, Remo barra, Press militar, Curl martillo, Plancha isométrica).
- Estado `videosTecnica` y funciones de acción `agregarVideoTecnica` y `eliminarVideoTecnica` en el contexto global.
- **Página de Videoteca (`app/videoteca/page.tsx`):**
- Interfaz con estética oficial del gimnasio (fondo oscuro slate-950, acentos azul eléctrico, tipografía audaz).
- Filtros por píldoras de grupo muscular y buscador en vivo.
- Modal de reproducción con visor de YouTube y lista de verificación biomecánica.
- Modal administrativo para alta de nuevos tutoriales.
- **Navegación y Enlaces (`components/sidebar.tsx` y `app/page.tsx`):**
- Se aseguró la ruta `/videoteca` en el menú lateral de ambos roles.
- Se enlazó el carrusel de videos del Home del alumno con la videoteca.
- **¿Dónde se hizo?**
- `lib/types.ts`: Definición de la interfaz `VideoTecnica`.
- `lib/mock-data.ts`: Mock de videos con tips y enlaces de técnica.
- `lib/store.tsx`: Manejo del estado global de videos y métodos `agregarVideoTecnica` / `eliminarVideoTecnica`.
- `app/videoteca/page.tsx`: Pantalla completa de la Videoteca y modal reproductor.
- `components/sidebar.tsx` y `app/page.tsx`: Enlaces directos a la videoteca y a las rutinas.

10. Correcciones Generales de Front y Módulo de Planes Dinámicos (Propuesta de Bruno)

- **Consulta y Requerimiento de Bruno:**

“"Habría que hacer una opción en configuración para que él pueda poner los planes, porque eventualmente van a cambiar los precios o cualquier cosa, que lo haga él directamente, no sé si es muy difícil."”

- **Orden de Lucas:**

“"Solucioná todo eso primero" (refiriéndose a los errores del front detectados: modal de alumno que no guardaba, 404 de catálogo, links erróneos, métricas fijas y planes hardcodedos).”

- **Lógica de Negocio Adoptada:**

- **Planes dinámicos y editables:** En un contexto inflacionario o de cambios de propuesta comercial, el dueño del gimnasio necesita ajustar precios, agregar pases promocionales o pausar planes viejos sin depender de desarrolladores ni tocar código.

- **Integración total en formularios:** Los planes dejan de estar escritos "a mano" en los selectores. Al crearse o editarse un plan en `/planes`, se actualiza automáticamente en el modal de alta de alumnos del Home y en el panel de Alumnos.

- **¿Qué se corrigió y qué se agregó?**

1. **Módulo de Planes (`/planes`):**

- Nueva interfaz en `app/planes/page.tsx` con listado de membresías, precios por mes, frecuencia semanal y estado activo/pausado.
- Modal interactivo para crear nuevos planes o modificar precios existentes con impacto inmediato.
- Reemplazo en el menú lateral de la ruta fantasma `/catalogo` (que daba 404) por la nueva ruta oficial `/planes`.

2. **Persistencia y Store (`lib/types.ts` y `lib/store.tsx`):**

- Definición de la entidad `Plan`.
- Métodos en el store: `agregarPlan()`, `actualizarPlan()` y `eliminarPlan()`.
- Sincronización automática de `planes` en almacenamiento local (`localStorage`).

3. **Arreglo del Modal "+ Nuevo Alumno" en Home (`app/page.tsx`):**

- Se conectó `agregarAlumno(...)` en el evento `onSubmit`. Ahora los alumnos cargados desde el botón principal del Home se guardan efectivamente en la lista general.
- Se reemplazaron las opciones fijas del selector de planes por las opciones reales y dinámicas leídas del store.

4. **Corrección de Enlaces y Navegación:**

- En `components/sidebar.tsx`, el logotipo de ATLAS ahora dirige a `/` (Home) para todos los roles, permitiendo que el alumno regrese a su panel sin ser forzado a `/finanzas`.
- En `app/page.tsx`, el botón "Ver todas" de la tarjeta "Rutinas activas" ahora dirige correctamente a `/rutinas` en lugar de `/finanzas`.

5. Métricas Reales en el Home del Administrador:

- "Alumnos activos" ahora calcula la cantidad real de alumnos con `activo: true`.
- "Cuotas pendientes" ahora totaliza en tiempo real la suma de cuotas con estado pendiente o vencido.
- "Pagos recientes" renderiza los últimos cobros registrados cronológicamente en Finanzas.

6. Rutina Asignada Real en el Home del Alumno:

- La tarjeta "Mi Rutina de hoy" consulta `getRutinaDeAlumno(usuario.alumnoId)` y muestra el nombre y ejercicios del plan que el profesor le asignó realmente a ese alumno.
- **¿Dónde se hizo?**
 - `lib/types.ts` : Definición de la interfaz `Plan`.
 - `lib/mock-data.ts` : Lista inicial `PLANES MOCK`.
 - `lib/store.tsx` : Métodos de gestión de planes en el contexto global.
 - `components/sidebar.tsx` : Cambio de ruta de catálogo a `/planes` y corrección de enlace raíz en el logo.
 - `app/planes/page.tsx` : Nueva pantalla de gestión de planes y tarifas.
 - `app/page.tsx` : Corrección del guardado en el modal, métricas dinámicas, enlace a rutinas y rutina real del alumno.
 - `app/alumnos/page.tsx` : Selectores de planes dinámicos en los modales de creación y edición.